



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P.O.O : **Java**



Chapitre

3

Syntaxe et éléments de base de Java

Sofiane HACHICHA
ISIMG 2025 - 2026



3- Les bases du langage

3.1 Les commentaires (1/5)



- **Tout programme (grand ou petit, simple ou complexe) contient (ou devrait contenir) des commentaires.**
- **Les commentaires ont pour but d'expliquer :**
 - Ce que le programme est censé faire,
 - Les conventions adoptées,
 - Toute autre information rendant le programme lisible pour soi-même et surtout pour autrui.
- **Java dispose de trois types de commentaires :**
 - Les commentaires mono-ligne,
 - Les commentaires multi-lignes,
 - Les commentaires pour documentation.



3- Les bases du langage

3.1 Les commentaires (2/5)



● Commentaires mono-ligne :

- Les commentaires mono-ligne débutent avec les symboles "//" et qui se terminent à la fin de la ligne.

```
// Ce programme imprime la chaîne  
// de caractères " bonjour " à l'écran  
...
```

- Ils représentent des commentaires courts qui tiennent sur une seule ligne.





3- Les bases du langage

3.1 Les commentaires (3/5)



● Commentaires multi-lignes :

- Un commentaire multi-ligne commence par les caractères `/*` et se termine par `*/`.

```
/* Ce programme imprime la chaîne  
de caractères "bonjour" à l'écran  
*/
```

- A l'intérieur de ces délimiteurs, toute suite de caractères est valide (sauf évidemment `*/`).



3- Les bases du langage

3.1 Les commentaires (4/5)



● Commentaires pour documentation :

- Ces commentaires, appelés aussi **commentaires Javadoc**, servent à documenter les classes que l'on définit.
- Ces commentaires sont encadrés entre `"/**"` et `"*/"`
`/**`
`* Documentation de la classe.`
`*/`
- Java exige que ces commentaires figurent **avant** la définition de la classe ou d'un membre de la classe qu'ils documentent.
- Ces commentaires servent à générer une documentation officielle sous forme de pages HTML à l'aide de l'outil **Javadoc**.



3- Les bases du langage

3.1 Les commentaires (5/5)



● Commentaires pour documentation :

- **Java 23** nous offre la possibilité d'écrire la Javadoc en Markdown pour sa mise en forme.

- Pour rester compatible avec les anciennes Javadoc, on utilisera un **triple slash** pour spécifier une ligne de Javadoc.

```
/**
 * Classe test
 * <p>
 * Un premier exemple :
 * <ul>
 * <li>simple
 * <li>complet
 * </ul>
 */
```

```
/// Classe test
///
/// Un premier exemple :
/// - simple
/// - complet
```



3- Les bases du langage

3.2 Variables

P.O.O : Java



● Le langage Java est un langage fortement typé :

- Il faut préciser le type de chaque variable
- Une variable peut avoir plusieurs valeurs différentes mais ne peut pas changer de type.

● Deux modes de typage en Java

- **Statique** : le type est précisé à la déclaration de la variable.
- **Dynamique** : la variable est déclarée avec le mot-clé **var** et son type est dynamiquement déterminé à partir de la valeur initiale (ce mode est disponible depuis **Java 11**).





3- Les bases du langage

3.3 Typage dynamique des variables

● Déclarer une **variable locale** avec le mot-clé **var**

- **var** nomVariable = valeurInitiale;

● Une **variable locale** déclarée avec le mot-clé **var**

- doit être initialisée : la déclaration et l'initialisation doivent être faites en même temps.
- a un type déterminé en fonction de sa valeur initiale
- ne peut pas changer de type

● Exemple :

```
var x = 1;
```

```
x = "test"; // Ceci génère une erreur à la compilation
```



3- Les bases du langage

3.4 Typage statique des variables

● Déclarer une variable

- **type** nomVariable;

● Deux choix possibles pour typer les variables en **Java**

- **Types primitifs** : ils commencent par une lettre en minuscule.

- Exemple : Les types numériques, le type booléen et le type caractère.

- **Types objets** (ou références) : ils commencent par une lettre majuscule.



3- Les bases du langage

3.4.1 Les types primitifs numériques (1/4)

P.O.O : Java



Les types numériques entiers

	byte	short	int	long
Taille en bits	8	16	32	64
Etendue	-128 ... 127	-32768 ... 32767	$-2^{31} \dots 2^{31}-1$	$-2^{63} \dots 2^{63}-1$



Le bit est utilisé pour décrire les tailles.

- Un bit ne peut prendre que deux valeurs (0 ou 1).
- n** bits ne peuvent définir que **2ⁿ** valeurs.
- Un **octet** ou **byte** en anglais = 8 bits.
- Un octet peut donc contenir **2⁸** (soit 256) valeurs distinctes.



3- Les bases du langage

3.4.1 Les types primitifs numériques (2/4)

P.O.O : Java



Les types numériques flottants

- De même que pour les types numériques entiers, il existe deux types pour les nombres **flottants** :
 - float,
 - double.
- la seule différence réside dans la taille utilisée pour stocker une valeur de ce type.

	float	double
Taille en bits	32	64
Exemple de valeur	3.25 f	3.25



3- Les bases du langage

3.4.1 Les types primitifs numériques (3/4)

POO : Java



- Depuis **Java 7**, il est possible de formater une variable de type numérique en utilisant le caractère '_' : pratique pour améliorer la lisibilité des grands nombres.
 - `int entier = 32_000;`
 - `float pi = 3.14_15f;`
 - `double monDouble = 123_567_891;`
- Le caractère '_' doit être placé entre deux caractères numériques : il ne peut pas être utilisé au début ou à la fin d'une déclaration, ni avant ni après un séparateur de décimal. Ainsi, les déclarations suivantes **ne sont pas valides** :
 - `double d = 123_.159;`
 - `int entier = _123;`
 - `int entier2 = 123_;`
- Depuis **Java 22**, il est possible d'utiliser le caractère '_' pour déclarer des **variables anonymes**. C'est parfois utile quand la déclaration de variable est obligatoire, mais que la variable est inutile.

3- Les bases du langage

3.4.1 Les types primitifs numériques (4/4)

POO : Java



● Il est permis de déclarer des variables numériques :

- en **octal** , en utilisant le préfixe « **0** » :

- `int entier = 255; //Peut s'écrire « int entier = 0377; »`

- `int entier = 20; //Peut s'écrire « int entier = 024; »`

- en **hexadécimal**, en utilisant le préfixe « **0x** » :

- `int entier = 255; //Peut s'écrire « int entier = 0xFF; »`

- `int entier = 20; //Peut s'écrire « int entier = 0x14; »`

● Depuis **Java 7**, il est possible d'utiliser la notation **binaire**, en utilisant le préfixe « **0b** » :

- `int entier = 0b1111_1111; //Peut s'écrire int entier = 255;`

- `int entier = 0b1000_0000_0000; //... int entier = 2048;`

- `int entier = 0b1000000000000; //... int entier = 2048;`





3- Les bases du langage

3.4.2 Le type primitif booléen

- Ce type est introduit en Java par le mot-clé **boolean**.
- Ce type est un vrai type booléen
 - il accepte seulement deux états :
 - l'un est nommé **true** :
 - Symbolise un état d'acceptation,
 - l'autre est nommé **false**,
 - Symbolise un état de réfutation.
- **Attention, contrairement du langage C, le type booléen en Java n'est pas un sous-type numérique :**
 - En **C** par exemple , la valeur 0 est considérée comme fausse et les autres valeurs entières comme vraies.





3- Les bases du langage

3.4.3 Le type primitif caractère

- Ce type, introduit par le mot-clé **char**, permet la gestion des caractères.
- Le codage **ASCII** utilise **8** bits et permet de représenter seulement **128** caractères.
- **Java** utilise le codage de caractères universel **Unicode** qui est une extension du codage **ASCII**.
 - Java utilise **UTF-16** : un encodage Unicode qui représente chaque caractère en utilisant **16 bits**.
- Les variables de type **char** sont codées sur **2 octets non signés**. Elles ont des valeurs allant de 0 à $2^{16} - 1$.

char car = 'a';

char x = 97; // 97 est le code caractère de la lettre **a** en UTF-16



3- Les bases du langage

3.4.4 Les types objets (1/2)

P.O.O : Java



- Tout type non primitif, est un type objet.
- Le rôle d'un type objet est de référencer une zone mémoire.
- Un type objet a un rôle analogue à celui des pointeurs du langage C .
- Les enveloppes (**Wrapper classes**) de types primitifs sont des types objets en Java.
 - **Byte** pour byte, **Short** pour short, **Long** pour long, **Integer** pour int, **Float** pour float, **Double** pour double, **Boolean** pour boolean et **Character** pour char.
- Une variable de type tableau est une référence vers un objet tableau.
- Il existe plusieurs autres types objets : String, Date, etc.



3- Les bases du langage

3.4.4 Les types objets (2/2)

POO : Java



● l'API Java fournit des **classes enveloppes** qui permettent d'envelopper la valeur d'un type primitif.

● Créer une instance d'un Wrapper soit en utilisant :

- son constructeur (@Deprecated(since="9", forRemoval=true))
- la méthode de classe **valueOf** (méthode recommandée).

```
Integer i = Integer.valueOf(2);
```

● Pour obtenir la valeur enveloppée, on fait appel à la méthode **xxxValue()**, xxx étant le type sous-jacent

```
int x = i.intValue();
```

● Les classes enveloppes (sauf Character) définissent une méthode de classe de la forme **parseXXX** pour convertir une chaîne de caractères en un type primitif (inversement **valueOf(x).toString()** ou **toString(x)**)

```
int val = Integer.parseInt("1");
```

- Une variable de type d'une des classes enveloppes peut avoir la valeur **null**. Ce cas signifie l'absence de valeur.





3- Les bases du langage

3.4.5 Déclaration et initialisation des variables

Exemples

- `int nbreDeMois = 12 ;` `int nbreDeMois = 4*3 ;`
- `int nbrDeDoigts = 012 ;` `int annee = 0x7E8;`
- `boolean test = false ;` `char unCaractere = 'c' ;`
- `float solde = 1.3f ;` `float salaire = 1.5e+3f ;`
- `double unDouble = 1.3 ;` `double nbreDouble = 5.8d ;`
- `byte b = Byte.parseByte("100");` `long q = 100_000_000_000l;`
- `float xd = Float.parseFloat("4f");`
- `Long l = new Long(5);` `Long y = Long.valueOf(100l);`
- `int x = 10;` `String s1 = Integer.valueOf(x).toString();`
- `double db = 4.1;` `String s2 = Double.toString(db);`
- `double z = 9.67d;` `short h = Double.valueOf(z).shortValue();`
- `String message = "bonjour" ;`

• Et éventuellement, un « **modificateur d'accès** » :

`final double pi=3.14159;`

String n'est pas un type primitif, mais une classe



3- Les bases du langage

3.5 Les opérateurs

P.O.O : Java



● En **Java**, les opérateurs sont regroupés par :

● type d'opérations :

- d'assignation
- arithmétique
- d'incrémentation et de décrémentation
- de comparaison
- de bits
- logique
- sur les chaînes de caractères
- de manipulations binaires

● le nombre d'opérandes :

- unaire
- binaire
- ternaire



3- Les bases du langage

3.5.1 Opérateurs unaires



Opérateurs unaires	Action	Exemple
-	négation	$i = -j$
++	incrément de 1	$i = j++$ ou $i = ++j$
--	décrément de 1	$i = j--$ ou $i = --j$

● ++ et -- peuvent **préfixer** ou **postfixer** la variable.

● $i = j++$: **post-incrément**

● La valeur en cours de j est affectée à i et ensuite la valeur de j est incrémentée de 1.

● $i = ++j$: **pré-incrément**

● La valeur en cours de j est incrémentée de 1 et ensuite la valeur de j est affectée à i .

Exemple :

```
int x = 2, y = x++;
System.out.println("x= "+x+"; y= "+y);
int z = ++x;
System.out.println("z= "+z);
```

Résultat d'affichage



3- Les bases du langage

3.5.2 Opérateurs binaires

P.O.O : Java



Opérateurs binaires	Action	Exemple	Syntaxe équivalent
+	addition	$i = j + k;$	
+=		$i += 2;$	$i = i + 2$
-	soustraction	$i = j - k;$	
-=		$i -= j;$	$i = i - j$
*	multiplication	$x = 2 * y;$	
*=		$x *= x;$	$x = x * x$
/	division (tronque si les arguments sont entiers)	$i = j / k;$	
/=		$x /= 10;$	$x = x / 10$
%	modulo	$i = j \% k;$	
%=		$i \% = 2;$	$i = i \% 2$
<<	décalage à gauche	$i < < k;$	$i * 2^k$
>>	décalage à droite	$i > > k;$	$i / 2^k$ (si $i \geq 0$)



3- Les bases du langage

3.5.3 Opérateurs relationnels

POO : Java



- Le résultat d'une comparaison est une valeur booléenne (**vrai** ou **faux**).
- Dans le langage **Java**, le résultat d'une comparaison est **true** ou **false**.

Opérateurs relationnels	Action	Exemple
<	plus petit que	x<i;
>	plus grand que	i>100;
<=	plus petit ou égal que	j<=k;
>=	plus grand ou égal que	c>='a';
==	égal à	i==20;
!=	différent de	c!='z';



3- Les bases du langage

3.5.4 Opérateurs logiques

POO : Java



Opérateurs logiques	Action	Exemple	Syntaxe équivalent
!	négation	!p	
&	ET	p & (i < 10)	
	OU	p q	
^	OU exclusif	p ^ false	
~	Complément	~p	
&&	ET évalué	p && q && r	
	OU évalué	p q r	
!=	négation assignée	p != q;	
&=	ET assigné	p &= q	p = p & q
=	OU assigné	p = q	p = p q

● Dans le cas d'un opérateur logique évalué, l'évaluation de l'expression logique est stoppée dès que sa valeur de vérité peut être assurée.



3- Les bases du langage

3.5.5 Opérateurs ternaires



- Un unique opérateur **ternaire**.

- Cette expression est une sorte de **si-alors-sinon** qui renvoie une valeur :

- $a = (\text{condition } e) ? x : y$

- si la condition e est vraie alors a vaut x sinon a vaut y .

- Exemples :

```
int res = (v==2) ? 1 : 0;
```

- On affecte à res la valeur 1 si v vaut 2 , sinon on affecte à res la valeur 0 .

```
String s = age >= 18 ? "majeur" : "mineur";
```

- On affecte à s la valeur **majeur** si age est supérieur ou égal à 18 , sinon on affecte à s la valeur **mineur**. 87





3- Les bases du langage

3.6 structures de contrôle

POO : Java



- Java propose des structures de contrôle classiques pour effectuer des instructions en fonction de la valeur de certaines variables.
- Java est rétro-compatible avec le C et possède quelques structures supplémentaires mais pas de goto.
- Les conditions des **if**, **for**, **while** doivent être des **booleans**.
- **Java 14**, **Java 21** et **Java 23** ont introduit plusieurs nouveautés dans la syntaxe du **switch**.
- Par convention, il est préférable d'utiliser un bloc même si il y a une seule instruction.



3- Les bases du langage

3.6.1 structures de contrôle : **if**

P.O.O : Java



● Syntaxe :

```
if (expression booléenne) {instruction ;}
```

● Exemple :

```
public class ifApplication1 {  
  
    public static void main(String[ ] args)  
    {  
        int i = 4 ;  
        if ( i % 2 != 0 ) {System.out.print (" i est impair ");}  
    }  
}
```

Résultat d'affichage



Attention : L'expression logique attendue est obligatoirement de type **boolean**.

- si l'on écrit **if (i = 1)**, le compilateur détectera une erreur, car le type de l'expression logique est alors **int**.

3- Les bases du langage

3.6.1 structures de contrôle : **if - else**

P.O.O : **Java**



Syntaxe	Exemple
<pre> if (expression booléene) { instruction 1 ; instruction i ; } else { instruction j ; instruction n ; } </pre>	<pre> public class if_elseApplication { public static void main(String[] args) { int i = 4 ; boolean test = (i % 2 == 0) ? true : false; if (test) { System.out.println (" i est pair "); } else { System.out.println (" i est impair "); } } } </pre>

Résultat d'affichage



3- Les bases du langage

3.6.1 structures de contrôle : **if – else if**

P.O.O : **Java**



Syntaxe	Exemple
<pre> if (expression booléene 1) { instruction 1 ; instruction i ; } else if (expression booléene 2) { instruction j ; instruction m ; } else { instruction m+1 ; } </pre>	<pre> public class if_else_ifApplication { public static void main(String[] args) { var x = -3; if (x > 0) { System.out.println (x + " est strictement positif"); } else if (x < 0) { System.out.println (x + " est strictement négatif"); } else { System.out.println (x + " est nul"); } } } </pre>
Résultat d'affichage	





3- Les bases du langage

3.6.2 structures de contrôle : **switch** (1/8)



- L'instruction "**switch**" permet d'effectuer une sélection parmi plusieurs valeurs (appelé aussi **Switch Statement**).
- Le **break** permet de quitter le **switch**.
- Le **default** est optionnel, il sera exécuté si la valeur de la variable ne correspond à aucune valeur de **case**.
- Avant Java 21, une **variable** dans **switch** peut être :
 - de type simple : **int**, **short**, **byte** et **char**
 - de type Wrapper correspondant : **Integer**, **Short**, **Byte** et **Character**
 - de type **énumération** depuis **Java 6** (**Exemple** :
enum Civilite {Monsieur, Madame, Mademoiselle};)
 - de type chaîne de caractère (**String**) depuis **Java 7**



3- Les bases du langage

3.6.2 structures de contrôle : **switch** (2/8)

P.O.O : **Java**



Sofiane HACHICHA
ISIMG 2025 - 2026

Syntaxe	Exemple
<pre> switch (nomVariable) { case valeur_1 : instr1_1; instr1_2; ... break; ... case valeur_N : instrN_1; instrN_2; break; default: /* optionnel */ instrD_1; instrD_2; ... break; } </pre>	<pre> public class switchApplication1 { public static void main(String[] args) { int i = 1; switch (i) { case 0 : System.out.println (" 0 "); break; case 1 : System.out.println (" 1 "); break; case 2 : System.out.println (" 2 "); break; default : System.out.println (i); break; } } } </pre>

Résultat d'affichage



3- Les bases du langage

3.6.2 structures de contrôle : **switch** (3/8)

P.O.O : **Java**



Sofiane HACHICHA
ISIMG 2025 - 2026

● Il est possible de regrouper plusieurs cases.

- Une case peut accepter plusieurs valeurs séparées par des virgules.

Résultat d'affichage

Exemple

```
public class switchApplication2 {

    public static void main(String[ ] args)
    {
        String x = "2";

        switch ( x )
        {
            case "1" :
                System.out.println ( " un " );
                break;
            case "2" :
                System.out.println ( " deux " );
            case "3", "4" :
                System.out.println ( " trois " );
                break;
            default :
                System.out.println ( " autre " );
                break;
        }
    }
}
```



3- Les bases du langage

3.6.2 structures de contrôle : **switch** (4/8)

P.O.O : **Java**

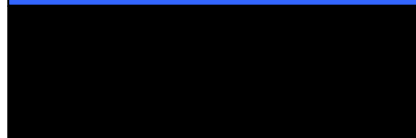


Sofiane HACHICHA
ISIMG 2025 - 2026

● Une nouvelle syntaxe du **switch** (depuis **Java 14**) :

- Remplacement de ":" par "->".
- Les instructions à droite de "->" dans chaque **case** peuvent être multiples et doivent être placées dans un bloc de code.
- Pas besoin de **break**

Résultat d'affichage



Exemple

```
public class switchApplication3 {

    public static void main(String[ ] args)
    {
        int seats = 1; String type = "rien";

        switch ( seats )
        {
            case 1, 2 -> type="small";
            case 3, 4, 5 -> { // accolade obligatoire
                System.out.println ( " test " );
                type="medium";
            }
        }

        System.out.println(type);
    }
}
```



3- Les bases du langage

3.6.2 structures de contrôle : **switch** (5/8)

P.O.O : **Java**

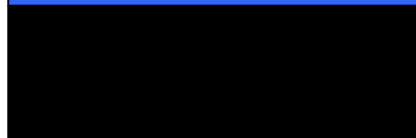


Sofiane HACHICHA
ISIMG 2025 - 2026

● Le **switch** peut aussi être utilisé comme une expression et retourner une valeur (appelé **Switch Expression**).

- La valeur retournée sera fournie à droite de "->".
- En présence d'un bloc de code, utiliser **yield** pour préciser la valeur à retourner.
- La clause **default** est obligatoire sauf pour les types énumérés

Résultat d'affichage



Exemple

```
public class switchApplication4 {

    public static void main(String[ ] args)
    {
        int seats = 5;

        var type = switch ( seats )
        {
            case 1, 2 -> "small";
            case 3, 4, 5 -> {
                System.out.println ( " test " );
                yield "medium";
            }
            default -> "big";
        }; // bloc default obligatoire sinon erreur
        System.out.println(type);
    }
}
```





3- Les bases du langage

3.6.2 structures de contrôle : **switch** (6/8)

POO : Java



Exemple

```
enum Jours {LUNDI, MARDI, MERCREDI, JEUDI, VENDREDI, SAMEDI, DIMANCHE}

public class switchApplication5 {

    public static void main(String[ ] args)
    {
        Jours jr = Jours.MERCREDI;

        int nbrLettres = switch (jr) {
            case LUNDI, MARDI, JEUDI -> {yield 5;}
            case SAMEDI -> 6;
            case DIMANCHE, MERCREDI, VENDREDI -> 8;
        };
        System.out.println("Nombre de lettres du jour : "+nbrLettres);
    }
}
```

Résultat d'affichage





3- Les bases du langage

3.6.2 structures de contrôle : **switch** (7/8)

POO : **Java**



- Depuis **Java 21**, il est possible de vérifier directement le type d'une variable non primitif avec un switch statement.
 - Switch Pattern Matching
- Grâce à la JEP 455 de **Java 23** (en mode preview), il est désormais possible d'appliquer des vérifications aux types primitifs.
 - Il est ainsi envisageable de vérifier de manière concise si un type primitif est en mesure d'être stocké dans un autre.





3- Les bases du langage

3.6.2 structures de contrôle : **switch** (8/8)

POO : Java



Exemple

```
public class TestSwitch{
    static void validObject(Object obj){
        switch (obj) {
            case Integer i -> System.out.println(i+1);
            case Double d -> System.out.println(d+2);
            case null, default -> System.out.println(obj); }
    }
    static void validPrimitif(long val){
        switch (val) {
            case short s -> System.out.println(val + " instanceof short " + s);
            case long l -> System.out.println(val + " instanceof long : " + l);
            case double d -> System.out.println(val + " instanceof double : " + d); }
    }
    public static void main(String[] args) {
        var x = 1000;
        validObject(x);
        validPrimitif(x); }
}
```

Résultat d'affichage



3- Les bases du langage

3.6.3 structures de contrôle : **for** (1/3)

P.O.O : Java



Exemple :

```
public class forApplication {

    public static void main(String[ ] args)
    {
        int somme = 0;
        for ( int compteur = 1, max = 4 ; compteur < max ; compteur++)
        {
            somme = somme + compteur;
        }
        System.out.println (somme );
    }
}
```

Résultat d'affichage

Fonctionnement :

- initialisation du compteur,
- comparaison avec max (expression booléenne),
- réalisation des instructions,
- Incrémentation du compteur et on recommence.



3- Les bases du langage

3.6.3 structures de contrôle : **for** (2/3)

P.O.O : **Java**



- Impossible d'omettre l'initialisation, l'expression booléenne ou l'incrément dans la déclaration d'un **for**. Cependant, il est possible de les laisser vides en plaçant correctement un ou plusieurs « ; ».

```
int i = 0;
for (; i < 10; ++i) {
    // instructions
}
```

- Du coup, il est simple de réaliser une boucle infinie :

```
for (int j=1; j < 5;) {
    // instructions
}
```

```
for (; ;) {
    // instructions
}
```





3- Les bases du langage

3.6.3 structures de contrôle : **for** (3/3)

P.O.O : **Java**



- Il existe une forme améliorée du **for** (appelée **for-each**) spécialement conçue pour parcourir des collections d'éléments (un tableau par exemple).

```
for (int i : maCollection) {  
    // instructions à exécuter  
}
```

- Il faut que le type de la variable **i** soit compatible pour l'assignation d'un élément de la collection (un tableau d'entiers (maCollection) dans l'exemple précédent) .





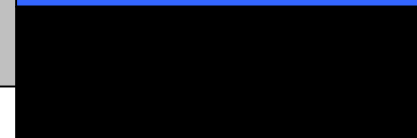
3- Les bases du langage

3.6.4 structures de contrôle : **while**

● Exemple :

```
public class whileApplication {  
  
    public static void main(String[ ] args)  
    {  
        int compteur = 1 ; int max = 4 ; int somme = 0;  
  
        while (compteur < max )  
        {  
            somme = somme + compteur ;  
            compteur++;  
        }  
  
        System.out.println (somme );  
    }  
}
```

Résultat d'affichage





3- Les bases du langage

3.6.5 structures de contrôle : **do - while**

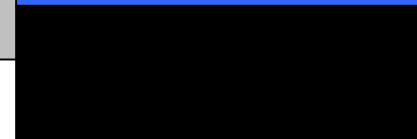
P.O.O : **Java**



● Exemple :

```
public class do_WhileApplication {  
  
    public static void main(String[ ] args)  
    {  
        int compteur =1; int max = 4 ; int somme = 0;  
  
        do  
        {  
            somme = somme + compteur ;  
            compteur++;  
        }  
        while (compteur < max ) ;  
  
        System.out.println (somme );  
    }  
}
```

Résultat d'affichage



3- Les bases du langage

3.6.6 structures de contrôle : **break**



● Utilisation :

- Pour sortir d'une structure de contrôle (**for** ou **while** ou **do-while**) avant que la condition du test soit remplie.
- Quand la boucle rencontre une instruction **break**, elle se termine immédiatement sans exécuter le code restant.

```
public class breakApplication {

    public static void main(String[ ] args)
    {
        int compteur =0;

        while (compteur < 10 )
        {
            System.out.println (compteur );
            compteur++;

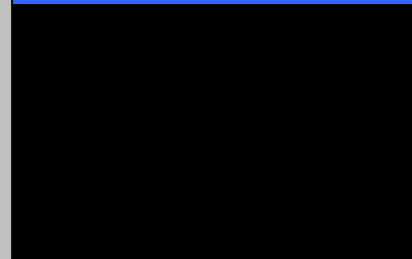
            if (compteur == 5) break;

        }

    }

}
```

Résultat d'affichage



3- Les bases du langage

3.6.7 structures de contrôle : **continue**

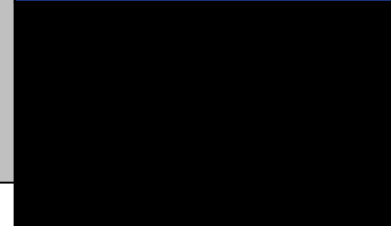
● Utilisation :

- Pour arrêter l'exécution de l'itération actuelle d'une structure de contrôle (**for** ou **while** ou **do-while**) et de reprendre l'exécution à l'itération suivante.

```
public class continueApplication {

    public static void main(String[ ] args)
    {
        int compteur =0;
        for(compteur=0 ; compteur< 6 ; compteur++ )
        {
            if (compteur == 4) continue;
            System.out.println (compteur );
        }
    }
}
```

Résultat d'affichage



3- Les bases du langage

3.6.8 structures de contrôle : **return**

P.O.O : **Java**



- Le type de données de la valeur retournée doit correspondre au type de retour de la méthode.

```
public double getSalaire() {  
    return 1500.7; }
```

- Quand une méthode est déclarée **void**, il faut utiliser la forme de retour qui ne retourne pas de valeur : **return;**

```
public void doSomething(){  
    return; }
```

- Écrire des instructions immédiatement après une instruction **return** génère une erreur de compilation puisque ces instructions ne seront jamais exécutées.

```
if (i % 2 == 0) {  
    return 0;  
    i++; // Erreur à la compilation  
}
```



3- Les bases du langage

3.7 Les conversions des types (1/12)



● Il y a 2 catégories de conversions possibles :

● Conversions **explicites** :

- celles faites sur une demande explicite par le programmeur appelées le transtypage (ou cast).

● Conversions **implicites** :

- celles faites automatiquement par le compilateur :

- lors d'une affectation,
- lors d'une promotion arithmétique,
- lors d'un passage de paramètres (lors de l'invocation d'une méthode),
- Lors d'une conversion du type primitif vers son Wrapper (ou inversement)



3- Les bases du langage

3.7 Les conversions des types (2/12)

P.O.O : Java



● Conversion explicite :

- Objectif :
 - changer le type d'une donnée si besoin.
- Comment ? :
 - Préfixer l'opérande par le type choisi.
 - Encadrer le type choisi par des parenthèses.

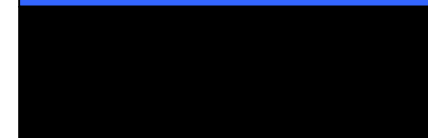
● Exemple :

double d = 2.5 ;

long l = **(long)** d ;

`System.out.println("d = "+d+" ; l="+l);`

Résultat d'affichage



3- Les bases du langage

3.7 Les conversions des types (3/12)

POO : Java



● Conversion implicite lors d'une affectation :

● Objectif :

- changer automatiquement le type d'une donnée si besoin.

● Comment ? :

● Exemple 1 :

- Contrairement à ce qui se passe dans **C**, les conversions pouvant provoquer une perte de valeur ne sont pas autorisées de manière implicite comme dans **C** : cela doit être explicitement demandé.

● Exemple 2 :

- Un type **char** peut être utilisé partout où une valeur de type **int** est permis.



3- Les bases du langage

3.7 Les conversions des types (4/12)

P.O.O : Java



● Conversion implicite lors d'une affectation :

- Illustration pour l'exemple 1:

```
public class ProgrammeAffectation {

    public static void main(String[ ] args)
    {
        int i;
        short j = 2;
        i = j;
        float k = 1.2f;
        i = k;
    }
}
```

Conversion implicite

Erreur de compilation

- Solution :



3- Les bases du langage

3.7 Les conversions des types (5/12)

● Conversion implicite lors d'une affectation :

- Illustration pour l'exemple 2:

```
public class ProgrammeSoustraction {

    public static void main(String[ ] args)
    {
        int i;
        i = 'A';
        System.out.println( i );
    }
}
```

← Conversion implicite

Résultat d'affichage



3- Les bases du langage

3.7 Les conversions des types (6/12)

POO : Java



● Conversion implicite lors d'une promotion arithmétique :

- Objectif :
 - Si un opérateur s'applique sur deux opérandes de types différents, un des deux opérandes sera converti (ou promu) au type de l'autre.
- Exemple :

```
public class ProgrammeSoustraction {

    public static void main(String[ ] args)
    {
        int i;
        i = 'A' - 1;
        System.out.println( i );
    }
}
```



Conversion implicite

Résultat d'affichage



3- Les bases du langage

3.7 Les conversions des types (7/12)

- Conversion implicite lors d'un passage de paramètres par l'invocation d'une méthode :

```
public class ProgrammeSoustraction {
```

```
    static short entierShort;
```

```
    static void Afficher( int entier )
```

```
    {
```

```
        System.out.print( entier );
```

```
    }
```

```
    public static void main(String[ ] args)
```

```
    {
```

```
        entierShort = 2 ;
```

```
        Afficher(entierShort) ;
```

```
    }
```

```
}
```



Invocation de la
méthode Afficher

P.O.O : Java

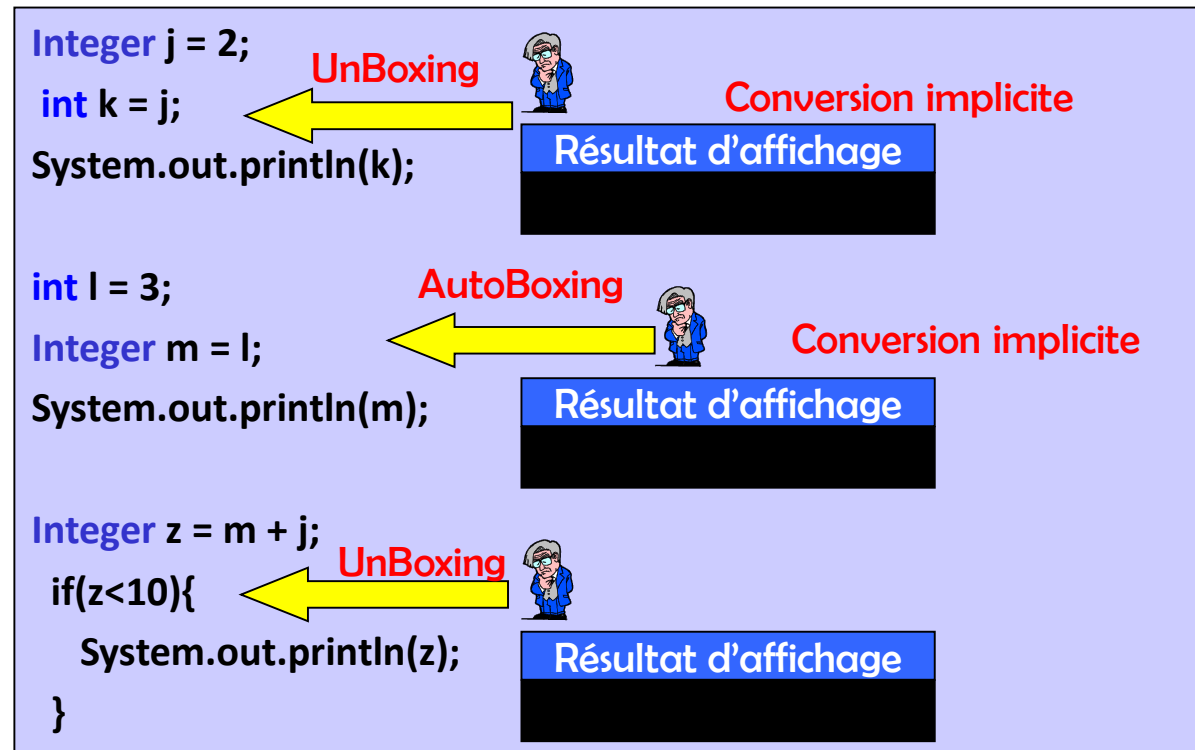


3- Les bases du langage

3.7 Les conversions des types (8/12)

- **Conversion implicite : passage d'un type primitif vers son Wrapper (ou inversement) depuis Java 5**

P.O.O : **Java**





3- Les bases du langage

3.7 Les conversions des types (9/12)

P.O.O : Java



● Passage d'un type primitif <--> son Wrapper

```
class Boxing1{  
    static void m(Long l) {System.out.println("Long");}  
    public static void main(String args[]){  
        long a=10;  
        m(a);  
    }  
}
```

Résultat d'affichage

```
class Boxing2{  
    static void m(int i) {System.out.println("int");}  
    public static void main(String args[]){  
        Integer s=30;  
        m(s);  
    } }  
}
```

Résultat d'affichage



3- Les bases du langage

3.7 Les conversions des types (10/12)

POO : Java



● Passage d'un type primitif <--> son Wrapper

```
class Boxing3{
    static void m(long l) {System.out.println("long");}
    public static void main(String args[]){
        Integer a=10;
        m(a);
    }
} // Unboxing puis Widening → ☒ VALIDE
```

Résultat d'affichage

```
class Boxing4{
    static void m(Long l) {System.out.println("Long");}
    public static void main(String args[]){
        int a=10;
        m(a);
    }
} // Widening puis Autoboxing → ✗ NON VALID
```

← // Erreur à la compilation





3- Les bases du langage

3.7 Les conversions des types (11/12)

P.O.O : Java



● Passage d'un type primitif <--> son Wrapper

```
class Boxing5{  
    static void m(int i) {System.out.println("int");}  
    static void m(Short i) {System.out.println("Short");}  
    public static void main(String args[]){  
        short s=30;  
        m(s);  
    } }
```

Résultat d'affichage

```
class Boxing6{  
    static void m(long i) {System.out.println("long");}  
    static void m(short i) {System.out.println("short");}  
    public static void main(String args[]){  
        var a=10;  
        m(a);  
    } }
```

Résultat d'affichage



3- Les bases du langage

3.7 Les conversions des types (12/12)

P.O.O : Java



Attention :

- Il n'y a pas de conversion possible (implicite ou explicite) entre un type **entier** et le type **booléen** :

```
int i = 0;

if (i)
{
    .....
}

if (i != 0)
{
    .....
}
```

 // Erreur à la compilation

 // Ok



3- Les bases du langage

3.8 Les tableaux (1/4)



● Déclaration

- monodimensionnel

● `int[ ] tableau_Entier;`

● équivalent à : `int tableau_Entier[ ];`

● `int[ ] tb1, tb2;`

- multidimensionnel

● `Color rgb_cube[ ][ ][ ];`

● Création et initialisation

● `int[ ] primes = {1, 2, 3, 5, 7};`

● `int[ ] tableauEntier = new int[ ] {1, 2, 3, 4};`

● `String[ ] tabChaine = new String[ ] {"B", "le", "mot"};`

● `int[ ] tableau_Entier = new int[50];`

● `int tb[ ] = new int[10], value = 10;`

3- Les bases du langage

3.8 Les tableaux (2/4)

P.O.O : Java



- Les éléments d'un tableau de types primitifs en Java sont initialisés à une valeur par défaut en fonction du type de données.
- Les éléments d'un tableau d'objets en Java, par défaut, sont initialisés à **null**.
- Les indices des tableaux commencent par **0**.
- Exemple d'utilisation des tableaux :

```
public class tableauApplication {  
  
    public static void main(String[ ] args)  
    {  
        int[ ] tableau_Entier = new int[50];  
        System.out.println(tableau_Entier.length);  
        tableau_Entier[51]=10;  
    }  
}
```

Résultat d'affichage



3- Les bases du langage

3.8 Les tableaux (3/4)



- En Java, les tableaux sont statiques, ce qui signifie que leur taille ne peut pas changer une fois qu'ils ont été créés.
- Pour parcourir un tableau **tab** (de chaînes de caractères par exemple) déjà créé :

```
for (int i = 0; i < tab.length; i++){  
    System.out.println(tab[i]);  
}
```

Où encore avec un for amélioré (**for-each**)

```
for (String elt : tab){  
    System.out.println(elt);  
}
```



3- Les bases du langage

3.8 Les tableaux (4/4)

POO : Java



- Il est possible d'initialiser des tableaux à plusieurs dimensions.

```
int[][] tableauDeuxDimensions = {{1, 2}, {3, 4}};  
int[][][] tableauTroisDimensions = new int[2][10][5];
```

- Les tableaux multidimensionnels sont en fait des tableaux de tableaux

- Il est possible de déclarer un tableau sans préciser les dimensions au-delà de la première et affecter ensuite des tableaux à chaque élément du tableau principal. Ces **tableaux internes** peuvent même avoir des **tailles différentes**.

```
int[][] tableauDeuxDimensions = new int[2][];  
  
tableauDeuxDimensions[0] = new int[10];  
tableauDeuxDimensions[1] = new int[5];
```